Contents

1 PROVISIONAL PATENT APPLICATION SPECIFICATION 1
1.1 DRAFT — FOR ATTORNEY REVIEW ONLY . . . 1
1.2 Title of the Invention . . . 2
1.3 Cross-Reference to Related Applications . . . 2
1.4 Field of the Invention . . . 2
1.5 Background of the Invention . . . 2
1.5.1 The Problem of API Monetization . . . 2
1.5.2 Prior Art . . . 3
1.5.3 Unmet Need . . . 4
1.6 Summary of the Invention . . . 4
1.7 Brief Description of the Drawings . . . 6
1.8 Detailed Description of Preferred Embodiments . . . 6
1.8.1 1. System Overview and Terminology . . . 7
1.8.2 2. Pre-Funded Balance Monetization (Claim Family 1) . . . 8
1.8.3 3. Composable Constraint Engine (Claim Family 2) . . . 13
1.8.4 4. Hierarchical Trust Chain with Self-Similar Royalty Distribution (Claim Family 3) . . . 17
1.8.5 5. Nostr-Based Identity and Credential Exchange (Claim Family 4) . . . 23
1.8.6 6. Network Governance as Economic Defense (Claim Family 5) . . . 25
1.8.7 7. AI-Assisted Pricing Campaign Design Tool (Claim Family 6) . . . 28
1.9 Abstract . . . 30
1.10 Appendix A: Implementation References . . . 31
1.11 Appendix B: Cryptographic Primitives . . . 32
1.12 Appendix C: Constraint Engine Class Hierarchy . . . 33
1.13 Appendix D: Runtime Pricing Model Architecture . . . 33

1 PROVISIONAL PATENT APPLICATION SPECIFICATION

1.1 DRAFT — FOR ATTORNEY REVIEW ONLY

Legal Disclaimer: This specification was prepared by the inventor with AI assistance (Claude, Anthropic). It requires review by a registered patent attorney or agent before filing with the United States Patent and Trademark Office. This document does not constitute legal advice. Claude is not a lawyer.

Filing Target: Mid-2026 as provisional application (~$320 micro-entity fee)

Prior Art Disclosure Date: February 17, 2026 (public repository at `github.com/lonniev/thebra mcp` with GPG-signed tag `v1.0.0-prior-art` dated February 16, 2026)

1.2 Title of the Invention

System and Method for Pre-Funded Balance Monetization of Tool-Based APIs with Composable Constraint Pricing, Hierarchical Trust Chains, Cryptographic Identity, and Network Governance

---

1.3 Cross-Reference to Related Applications

None. This is the initial provisional filing.

1.4 Field of the Invention

The present invention relates generally to methods and systems for monetizing application programming interfaces (APIs), and more particularly to a system for monetizing tool-based API services through pre-funded credit balances settled via cryptocurrency micropayment networks, with composable pricing constraints, hierarchical multi-party revenue distribution, decentralized cryptographic identity, and community governance mechanisms.

1.5 Background of the Invention

1.5.1 The Problem of API Monetization

Application programming interfaces (APIs) are the primary mechanism through which software systems exchange data and invoke functionality. The emergence of AI agent frameworks — particularly the Model Context Protocol (MCP) specification for connecting AI assistants to external tools — has created a new class of API consumer: autonomous software agents that invoke dozens or hundreds of tool calls per session without human intervention at each step.

Traditional API monetization approaches fall into three categories, each with significant limitations in the AI agent context:

Subscription models require the API consumer to commit to recurring payments regardless of actual usage. This "pestering" of the customer — demanding payment decisions before value is received — creates friction that discourages adoption and penalizes intermittent users. Subscription management overhead (billing cycles, plan tiers, overage charges, cancellation flows) consumes engineering resources unrelated to the core service.

Per-request payment negotiation models (exemplified by the L402 protocol and the x402 protocol, described below) require a payment interaction at the point of service. When a client sends a request, the server responds with HTTP status 402 (Payment Required) along with payment instructions. The client must then construct a payment, transmit it, await confirmation, and resubmit the original request with proof of payment. This challenge-response pattern introduces latency,

interrupts tool call chains, and requires the client to maintain payment channel state alongside application state.

Marketplace aggregation models (exemplified by platforms such as MCPize, Apify, and Masumi Network) interpose a centralized marketplace between API providers and consumers. The marketplace handles billing, typically through conventional payment processors (Stripe, credit card networks). These platforms extract significant revenue shares (15% or more), require providers to conform to marketplace terms, and introduce single points of failure. None provide the provider with direct control over pricing policy or customer relationships.

1.5.2 Prior Art

1.5.2.1 L402 Protocol (Lightning Labs, 2020) The L402 protocol (originally named LSAT) was published by Lightning Labs in March 2020. It defines a mechanism for HTTP-native API payments using the Lightning Network. When a server receives an unauthenticated request, it returns HTTP 402 with a `WWW-Authenticate` header containing a Lightning invoice and a macaroon (a bearer credential with attenuable caveats). The client pays the invoice, receives a preimage, and combines the preimage with the macaroon to form an L402 token that authenticates subsequent requests.

L402 is implemented through Aperture, a reverse proxy that intercepts requests, verifies L402 tokens, and forwards authenticated requests to the upstream service.

L402 limitations relevant to the present invention:

- Every request that lacks a valid token triggers a payment negotiation cycle
- No mechanism for pre-funding a balance that covers multiple future requests
- Pricing rules are configured ad hoc in the reverse proxy; no composable constraint system exists
- Single-party architecture with no multi-layer revenue distribution
- Identity is bound to macaroons, not to a portable cryptographic keypair
- No community governance or network coordination mechanism

The L402 specification mentions "top-up" functionality in passing but does not define a pre-funded balance architecture. It mentions "surge pricing" as a possibility but provides no mechanism for implementing it.

1.5.2.2 x402 Protocol (Coinbase, 2025) The x402 protocol was launched by Coinbase in late 2025, with a partnership announced with Cloudflare in December 2025 for integration into Cloudflare's infrastructure. x402 defines a stablecoin-based (USDC) payment mechanism embedded in HTTP headers. The server responds with 402 and a JSON payment requirements object; the client constructs a payment payload, signs it with a wallet key, and includes it in a `PAYMENT-SIGNATURE` header on the retry request. A "facilitator" service (hosted by Coinbase) verifies the signature and settles the payment on-chain.

x402 limitations relevant to the present invention:

- Same challenge-response pattern as L402 — no pre-funded balance

- Two payment "schemes" only: "exact" (fixed price) and "upto" (variable with ceiling); no composable constraint pipeline
- Relies on a centralized Facilitator service operated by Coinbase
- Identity is a blockchain wallet address, not a portable network identity
- No hierarchical trust chain or revenue distribution mechanism
- No governance layer — Coinbase controls the Facilitator and the Foundation
- Uses stablecoins (USDC) pegged to fiat currency, not native cryptocurrency

1.5.2.3 MCP Monetization Platforms (2025-2026) Several platforms have emerged to monetize MCP tool services:

- **MCPize**: Marketplace model with 85% revenue share, Stripe-based payments
- **PayMCP/Walleot**: Decorator-based tool pricing (`@price`) with multiple payment modes via a centralized wallet service
- **Masumi Network**: Blockchain-based (Cardano-adjacent) payment integration
- **Moesif**: Traditional API analytics and billing wrapping Stripe/Chargebee/Zuora
- **Apify**: MCP server marketplace with conventional payment rails

None of these platforms use the Lightning Network for settlement. None implement a pre-funded credit balance model. None have composable constraint-based pricing. None have hierarchical trust chains with royalty distribution. None use decentralized cryptographic identity. None have community governance mechanisms.

1.5.3 Unmet Need

There exists a need for an API monetization system that:

1. Eliminates per-request payment negotiation by allowing consumers to pre-fund a credit balance from which individual tool invocations are deducted
2. Provides API operators with a composable, programmable pricing policy engine rather than ad hoc price configurations
3. Supports multi-party revenue distribution through a hierarchical trust chain where each layer uses the same protocol, enabling franchise-like scaling
4. Uses portable, decentralized cryptographic identity that requires no personally identifiable information, email addresses, passwords, or OAuth tokens
5. Provides community governance mechanisms that make the network's economic value reside in participation rather than in proprietary code
6. Assists API operators in designing pricing policies through AI-guided structured interviews, with schema-aware validation, multi-provider adversarial review, and cryptographically authenticated deployment to live endpoints

---

1.6 Summary of the Invention

The present invention provides an integrated system and method for monetizing tool-based APIs through six interconnected architectural components:

First, a pre-funded balance monetization system in which API consumers purchase credit units ("api_sats") via Lightning Network micropayments, and individual tool invocations deduct from that balance according to per-tool pricing rules, eliminating per-request payment negotiation entirely.

Second, a composable constraint engine that evaluates a pipeline of pricing constraints on each tool invocation — including temporal windows, finite supply caps, periodic refresh limits, congestion-based surge pricing, ad valorem pricing proportional to response value, and promotional modifiers — to produce a final deduction amount. The constraint engine supports runtime-configurable pricing models: named bundles of per-tool prices and constraint pipelines that are persisted in a database, activated atomically (at most one active per operator), resolved at tool-call time through a TTL-cached resolver, and subject to graceful degradation on database failure.

Third, a hierarchical trust chain with self-similar royalty distribution, in which an Authority certifies purchase orders for downstream Operators using cryptographic signatures, and each layer in the hierarchy implements the same protocol pattern. An Authority is structurally an Operator whose customers happen to be other Operators. Revenue flows through the chain via certification fees. A configuration-time fee floor invariant ensures economic viability: each non-Prime Authority's local fee rate must be greater than or equal to its upstream Authority's fee rate, preventing configurations in which an Authority would lose money on every certification. The chain also accommodates Advocates — community utility services that provide shared infrastructure (such as OAuth2 callback collection) without monetization, discoverable by peer services through registry-based service resolution. The Authority provisions each Operator's persistence layer as an isolated database schema, and the Operator encrypts all stored data with a key derived from its own private key — ensuring that neither the Authority, the infrastructure provider, nor any other Operator can read another Operator's data. A bootstrap protocol enables an Operator to discover its full configuration from a single private key, eliminating deployment-time environment variable proliferation. Within each Operator's encrypted schema, credentials are further partitioned by (service, npub) — the Operator's own credentials occupy one partition while each patron's credentials (OAuth tokens, API keys, bearer hashes) occupy independent partitions keyed by the patron's npub. This enables a single Operator to serve multiple concurrent patrons, each with their own independently encrypted credential set, with credential restoration surviving process restarts without re-authentication.

Fourth, a Nostr-based identity and credential exchange system in which each participant is identified by a Nostr public key (npub), citizenship is verified through cryptographic signature challenges, and API credentials are exchanged through encrypted direct messages on the Nostr relay network — requiring no email, passwords, OAuth, or personally identifiable information.

Fifth, a network governance system implemented through version-controlled community registries on a collaborative source code platform (GitHub), in which membership, dispute resolution, tax rates, and governance rules are managed through pull requests, providing cryptographic auditability through the platform's Merkle-tree commit history.

Sixth, an AI-assisted pricing campaign design tool that conducts structured multi-stage interviews with Operators to co-design pricing models, validates proposed constraint pipelines against the server's schema, solicits adversarial second opinions from independent AI providers, and deploys approved campaigns to live MCP endpoints authenticated by Nostr operator proofs. The interview

methodology and pricing guidance are managed through a community-maintained system prompt, enabling the network to refine campaign design practices without application updates.

These six components operate as a unified system referred to herein as the "Tollbooth" architecture, governed by a voluntary community referred to as the "Don't Pester Your Customer" (DPYC) Honor Chain.

1.7 Brief Description of the Drawings

FIG. 1 — System architecture diagram showing the relationship between Authority, Operator, and Consumer tiers, including Lightning Network settlement, the community registry, and the Nostr identity layer.

FIG. 2 — Pre-funded balance lifecycle showing the sequence from credit purchase through Lightning invoice generation, settlement, balance crediting, per-tool deduction, tranche expiration, and balance top-up.

FIG. 3 — Constraint Engine evaluation pipeline showing how a tool invocation traverses a stack of composable constraints to produce a final price, including the PriceModifier accumulation logic.

FIG. 4 — Trust chain certification flow showing the cryptographic handshake between Authority, Operator, and payment processor, including certificate generation, signature verification, and royalty extraction.

FIG. 5 — Nostr identity and credential exchange sequence showing the Secure Courier protocol for privacy-preserving credential delivery using NIP-44 encrypted direct messages on the Nostr relay network.

FIG. 6 — Network governance data flow showing the relationship between the community registry, the Oracle service, and the runtime membership verification performed during purchase certification.

FIG. 7 — AI-assisted pricing campaign design workflow showing the six-stage structured interview, machine-parseable progress extraction, client-side constraint pipeline validation, multi-provider adversarial second opinion, and NIP-98- authenticated deployment to the Operator's live MCP endpoint.

(Drawings to be prepared as formal patent figures from the Mermaid source diagrams accompanying this specification.)

1.8 Detailed Description of Preferred Embodiments

The following description presents the preferred embodiments of the invention in sufficient detail to enable a person skilled in the art of distributed systems, API design, and cryptocurrency payment protocols to reproduce the system.

1.8.1 1. System Overview and Terminology

The system comprises the following principal actors and components:

Consumer — An end user or AI agent that invokes tools on an Operator's MCP service. The Consumer holds a credit balance denominated in api_sats and is identified by a Nostr npub.

Operator — A service provider that exposes one or more tools through the Model Context Protocol (MCP). The Operator runs the Tollbooth middleware, which gates tool access behind a credit balance check. The Operator holds a BTCPay Server store and a Lightning Network node (or connection to one) for receiving payments. In the preferred embodiment, BTCPay Server is the payment processor. The OperatorRuntime abstracts payment processing behind an `ensure_btcpay()` method that constructs a payment client from vault-stored credentials, enabling future substitution of alternative Lightning-capable payment processors without changes to the credit purchase flow. The Operator is identified by a Nostr npub and is a registered member of the DPYC Honor Chain.

Advocate — A community utility service that provides shared infrastructure to peer MCP servers without monetization. Examples include OAuth2 callback collectors, relay aggregators, and shared credential vaults. The Advocate is registered in the community registry with a `services[]` array that enables peer discovery via registry-based service name resolution. The Advocate is identified by a Nostr npub and is a registered member of the DPYC Honor Chain, sponsored by the First Curator.

Authority — A certification service that signs purchase orders for downstream Operators. The Authority is structurally an Operator whose tools include the `certify_credits` tool. The Authority collects a certification fee (functionally equivalent to a tax or royalty) on each purchase order it signs. The Authority is identified by a Nostr npub and is a registered member of the DPYC Honor Chain.

First Curator — The root Authority in the trust chain. The First Curator self-signs its own certification events. The First Curator's npub is recorded in the community registry under version control. There is exactly one First Curator at any time.

api_sats — The internal unit of account for credit balances. api_sats are purchased with real Bitcoin satoshis via Lightning Network invoices. In the preferred embodiment, the exchange rate is 1:1 (one satoshi equals one api_sat). Volume incentives and promotional multipliers are implemented through the Constraint Engine (Section 3) rather than a fixed exchange-rate multiplier. api_sats have no value outside the Operator's service and are not redeemable for Bitcoin.

Tollbooth — The middleware library (distributed as a Python wheel, `tollbooth-dpyc`) that an Operator integrates into their MCP server. The library provides an `OperatorRuntime` class that the Operator instantiates once. The runtime's `register_standard_tools()` method registers 25+ standard tools (credit purchase, balance check, constraint evaluation, credential exchange, onboarding status, pricing management, Oracle delegation, and more). The Operator writes only domain-specific tools.

Credential Template — A declarative specification of the secrets an Operator or patron must provide. The `operator_credential_template` defines secrets needed to operate the service (payment processor credentials, upstream API keys), delivered once at registration time

via the Secure Courier protocol. The `patron_credential_template` defines per-patron secrets (API keys for proxied services), delivered via Secure Courier or, for OAuth2-integrated services, acquired through a browser-based authorization flow.

Credential Card (ncred) — A portable, encrypted credential token (encoded with the `ncred1` bech32 prefix) that a patron can redeem directly at an Operator's service without relay polling. After successful credential delivery via the Secure Courier protocol (Section 5.3), the Operator constructs an ncred card and delivers it to the patron as a Nostr DM.

Identity Credential — A Nostr event of kind 30080, signed by the Operator, issued to a patron upon successful credential receipt. The identity credential serves as a portable, cryptographically verifiable attestation that the patron has an established relationship with the Operator's service.

Honor Chain — The voluntary community of Operators and Authorities, organized as a network society. Members agree to common principles including the use of Bitcoin and Lightning for all commerce, avoidance of storing personally identifiable information on customers, proper identification of upstream Authorities, and acceptance of community governance.

Community Registry — A JSON file (`members.json`) stored in a version-controlled repository on GitHub. The registry records each member's Nostr npub, role (Citizen, Advocate, Operator, Authority, or First Curator), status (active or banned), membership date, upstream Authority npub, and associated services. Peer MCP servers use registry-based service name resolution to discover Advocate services by name without hardcoded URLs or environment variables.

Oracle — A free, publicly accessible MCP service that provides onboarding guidance, governance document retrieval, member lookup, and citizenship verification for the Honor Chain. The Oracle charges no api_sats for its tools.

1.8.2 2. Pre-Funded Balance Monetization (Claim Family 1)

1.8.2.1 2.1 Architectural Distinction from Prior Art The fundamental architectural distinction between the present invention and all identified prior art (L402, x402, marketplace platforms) is the separation of the payment event from the service event.

In challenge-response systems (L402, x402), the payment and service events are temporally coupled: each service request either succeeds (bearer token present) or triggers a payment negotiation (402 response with invoice). The client must complete payment before receiving service. This coupling introduces latency proportional to the payment network's confirmation time on every unauthenticated request, and requires the client to maintain payment state alongside application state.

In the present invention, the payment event (purchasing api_sats) is decoupled from the service event (invoking a tool). The consumer purchases credits in advance. Each tool invocation is a simple deduction from an in-memory balance — a subtraction operation with sub-millisecond latency. The consumer experiences no payment ceremony during tool use.

This decoupling is expressed as: **the toll is already paid**. The consumer passes through the tollbooth without stopping.

1.8.2.2 2.2 Credit Purchase Flow

The credit purchase flow proceeds as follows:

1. **Certificate Request**: The Operator's `purchase_credits` tool receives a request specifying the desired amount in satoshis and a cryptographically signed certificate from the Operator's upstream Authority (described in Section 4). If no certificate is provided and the Operator does not require Authority certification, the purchase proceeds without certification.
2. **Certificate Verification**: If a certificate is provided, the Tollbooth middleware verifies the cryptographic signature against the Authority's known public key, validates the certificate's claims (amount, timestamp, anti-replay identifier), and extracts the net amount (original amount minus the Authority's certification fee).
3. **Invoice Generation**: The Tollbooth middleware calls the BTCPay Server's Greenfield API to create a Lightning Network invoice for the verified amount. The invoice includes metadata linking it to the consumer's identity (Nostr npub or OAuth-derived patron identifier).
4. **Lightning Settlement**: The consumer pays the Lightning invoice using any Lightning-compatible wallet. BTCPay Server confirms settlement.
5. **Balance Crediting**: Upon confirmed settlement, the Tollbooth middleware credits the consumer's ledger with api_sats equal to the paid amount (1:1 ratio). Volume incentives, if any, are applied through the Constraint Engine's campaign system (e.g., bulk_bonus constraints) rather than a credit-time multiplier. Credits are granted idempotently — calling `check_payment` multiple times for the same invoice ID results in credits being granted exactly once.
6. **Tranche Creation**: Each credit purchase creates a discrete "tranche" in the consumer's ledger. Each tranche records the purchase amount, the grant timestamp, and an expiration timestamp (configurable by the Operator, default 30 days). Tranches are consumed in first-in-first-out (FIFO) order. Expired tranches are not deducted from but are removed from the available balance.

1.8.2.3 2.3 Per-Tool Deduction

Each tool exposed by the Operator is annotated with a cost in api_sats via a cost table, and wrapped with a `paid_tool` decorator that encapsulates the entire payment lifecycle:

```
TOOL_COSTS = {"search_thoughts": ToolTier.READ}  # 1 api_sat

runtime = OperatorRuntime(tool_costs=TOOL_COSTS, ...)

@tool
@runtime.paid_tool("search_thoughts")
async def search_thoughts(query_text: str, npub: str = "") -> dict:
    """Full-text search across thought names and content."""
    return await brain.search(query_text)
```

The decorator eliminates per-tool payment boilerplate: the function body contains only domain

logic. The Operator writes no debit, rollback, or balance-warning code. When a consumer invokes a tool, the decorator and underlying middleware:

1. Validates the consumer's cryptographic identity (npub). There is no silent fallback to the Operator's own identity; an empty or malformed npub is rejected immediately.
2. Evaluates the Constraint Engine (Section 3) to determine the effective cost, which may differ from the base cost due to active constraints.
3. Checks whether the consumer's available balance (sum of non-expired tranches) is sufficient.
4. If sufficient, deducts the effective cost from the oldest non-expired tranche (FIFO) and permits the tool invocation to proceed.
5. If insufficient, returns an error indicating the required balance and providing a direct path to purchase additional credits. The tool invocation does not proceed.
6. On successful execution, increments both the hourly demand counter and the lifetime supply counter for the tool (feeding the SurgePricingConstraint and FiniteSupplyConstraint respectively, Section 3.2) and injects a low-balance warning into the result if the consumer's remaining balance is below a configurable threshold.
7. On execution failure (exception), automatically rolls back the debit — the consumer is not charged for failed invocations.
8. Records the deduction in the consumer's daily usage log, including the tool name, cost, and timestamp.

1.8.2.4 2.4 Ledger Persistence The consumer's ledger (balance, tranches, usage log, invoice history) is persisted in an encrypted vault. In the preferred embodiment, the vault is implemented as a server-side key-value store (NeonVault) backed by serverless PostgreSQL (Neon), accessed via SQL-over-HTTP API, that survives process restarts and serverless cold starts. An in-memory LRU cache provides sub-millisecond read performance, with write-behind flushing to the persistent store on a configurable interval. Optimistic concurrency control via a monotonically incrementing `version` column prevents lost-update anomalies under concurrent writes.

All ledger data is encrypted at rest using AES-256-GCM with a key derived from the Operator's Nostr private key via HKDF-SHA256 (see Section 4.6). The Operator's persistence tenant is isolated at the database schema level, provisioned by the Authority during registration (see Section 4.5). This two-layer isolation — schema separation and key-based encryption — ensures that neither the Authority, the database infrastructure provider, nor any other Operator can read the ledger data.

For multi-tenant Operators that maintain per-consumer domain sessions (e.g., OAuth tokens, API clients), the system provides a two-tier session management architecture: a generic in-memory session cache (`SessionCache[T]`) with configurable TTL expiry, and a patron session cache (`PatronSessionCache[T]`) that wraps the in-memory cache with automatic persistence to the encrypted vault. On cold start, the patron session cache transparently restores sessions from the vault using an Operator-supplied restore callback that constructs the domain-specific session object from stored credentials. The Operator need not implement cache expiry, vault persistence, or cold-start restoration logic; these concerns are handled by the shared runtime.

The in-memory LRU cache interacts with the container lifecycle of the hosting environment to pro-

duce favorable runtime characteristics. In the preferred embodiment, the hosting platform (Horizon / FastMCP Cloud) keeps containers warm for at least 10 minutes of idle time. During this warm window, every tool call hits only in-memory state — no network round trips are required for ledger lookup, constraint evaluation, or identity resolution. Empirical measurements confirm sub-100ms per-tool-call overhead within a warm session (including network round trip to the container), with warm burst latency averaging 80ms and graceful degradation to 181ms after 3 minutes of idle and 219ms after 10 minutes of idle. The bootstrap sequence (retrieving the Neon database URL from the Authority, initializing the vault, hydrating the LRU cache) runs once per cold start and is cached for the process lifetime. Cold starts occur only on initial deployment or after extended idle periods (empirically 15-30+ minutes).

The ledger is associated with the consumer's Nostr npub. Every patron-facing tool requires an explicit npub parameter; there is no silent fallback to the Operator's own identity. This explicit-identity requirement is a security invariant of the system. Ledger data is never exposed to the consumer in full — only summary views (current balance, usage statistics, invoice history) are returned through read-only tools.

1.8.2.5 2.5 Demurrage and Expiration Purchased api_sats are subject to expiration (demurrage). Each credit tranche carries an expiration timestamp. Upon expiration, the remaining api_sats in the tranche are removed from the consumer's available balance. This design choice serves three purposes:

1. **Prevents store-of-value accumulation**: Without expiration, consumers would treat pre-funded balances as deposits, creating an implicit fiduciary obligation for the Operator to maintain backward compatibility with historical protocol versions indefinitely.
2. **Encourages timely consumption**: Expiration creates a natural incentive for consumers to use purchased credits, driving API traffic and revenue.
3. **Simplifies ledger management**: Expired tranches can be pruned from the ledger, bounding storage growth.

The expiration period is configurable per Operator. The preferred embodiment uses a 30-day default.

When the Pricing Resolver finds no stored pricing model but the Operator's registered tools carry base costs, the system auto-generates a default pricing model from the tool cost annotations. This ensures every Operator has a valid pricing model from first deployment without requiring manual configuration.

1.8.2.6 2.6 Runtime Performance Characteristics — Per-Session vs. Per-Request The pre-funded balance model converts the payment protocol from a per-request (cold) interaction to a per-session (warm) interaction. In prior art systems (L402, x402), every API call is potentially a payment event: the server must issue an HTTP 402 redirect, the client must generate or retrieve a Lightning invoice or stablecoin payment, the payment network must confirm settlement, and the client must retry the original request with proof of payment. Each of these steps introduces network round trips and cryptographic verification overhead.

In the present invention, the payment ceremony occurs exactly once per session: the consumer

purchases credits (Section 2.2), and the bootstrap sequence initializes the in-memory ledger cache. Thereafter, N tool calls execute at in-memory speed. The critical path for each tool invocation within a warm session consists of: (1) an in-memory balance lookup from the LRU cache, (2) constraint pipeline evaluation against in-memory state, (3) identity resolution from the cached session context, and (4) a balance deduction with write-behind persistence. None of these steps require a network round trip.

Empirical measurements on the preferred hosting platform (Horizon / FastMCP Cloud) demonstrate the following per-tool-call latencies (including the full network round trip from the AI client to the container and back):

Condition	Measured Latency
Warm burst (consecutive calls)	~80ms average
After 3 minutes idle	~181ms
After 10 minutes idle	~219ms
Per-tool-call overhead within warm session	<100ms

The hosting platform maintains container warmth for at least 10 minutes of idle time. Within this window, all in-process state — the bootstrap result, the courier service for credential exchange, and the ledger LRU cache — persists across tool calls without reinitialization. This validates the optimistic caching design: hot-path tool calls are pure in-memory operations with zero network overhead for state resolution.

This architecture provides a structural latency advantage over per-request payment protocols. Where L402 or x402 would impose payment negotiation overhead on every API call, the Tollbooth system amortizes the single payment event across an arbitrarily long session of tool invocations, each completing in under 100ms.

1.8.2.7 2.7 Demand Tracking The OperatorRuntime provides demand tracking primitives — `get_global_demand()`, `fire_and_forget_demand_increment()`, and `fire_and_forget_supply_increment()` — that record aggregate invocation counts in the `tool_demand` table using the Neon vault. The table schema is `(tool_name TEXT, window_key TEXT, count INTEGER)` with a composite primary key. Hourly demand uses time-bucketed window keys (e.g. `"2026-04-20T14:00"`) and feeds the SurgePricingConstraint (Section 3.2). Lifetime supply uses a sentinel window key `"__total__"` and feeds the FiniteSupplyConstraint (Section 3.2) when configured with global scope. Both counters are incremented atomically via `INSERT ... ON CONFLICT DO UPDATE SET count = count + 1` and are incremented automatically by the `paid_tool` decorator on each successful invocation; Operators do not call them explicitly. Reads are on-demand at constraint evaluation time; increments are fire-and-forget asynchronous tasks that never block tool execution.

1.8.2.8 2.8 Upstream Payment Encapsulation (x402 Adapter) The SDK provides an optional HTTP client adapter (`X402Client`) that enables Operators to transparently consume upstream APIs protected by the Coinbase x402 payment protocol. When an upstream HTTP endpoint returns

a 402 status with a `payment-required` header, the adapter parses the payment requirements, signs an EIP-712 typed-data authorization using the Operator's agentic wallet private key, and retries the request with the signed payment in an `X-PAYMENT` header.

This encapsulation pattern treats upstream x402 fees as Operator cost of goods sold (COGS), analogous to server rental or bandwidth charges. Patrons interact exclusively through the Tollbooth pre-funded balance model — they never see the 402 handshake, USDC settlement, or Ethereum wallet mechanics. The Operator's agentic wallet credentials are delivered via Secure Courier (Section 4) and stored in the encrypted Neon vault.

The adapter is per-tool opt-in: Operators import and configure it only for tool handlers that access x402-protected upstreams. Tools that access non-x402 APIs use standard HTTP clients. The adapter is gated behind an optional dependency group (`[x402]`) and imposes no import cost on Operators who do not use it.

This architecture enables DPYC Operators to act as wholesale aggregators of x402-priced services, re-denominating upstream USDC costs into pre-funded sat-denominated tool calls. The Operator captures the customer relationship; x402 becomes a commodity upstream input.

1.8.3 3. Composable Constraint Engine (Claim Family 2)

1.8.3.1 3.1 Architecture The Constraint Engine is a pipeline evaluator that determines the effective cost and access permission for each tool invocation. It is the mechanism by which Operators implement programmable pricing policies without modifying application code.

The engine comprises:

- **ToolConstraint**: An abstract base class defining the interface for all constraints. Each constraint implements `evaluate(context) -> ConstraintResult` and `describe() -> str`.
- **ConstraintContext**: An immutable data structure containing the information available to constraints during evaluation: a `LedgerSnapshot` (current balance, total consumed, total deposited), a `PatronIdentity` (the consumer's identifier and associated metadata), and an `EnvironmentSnapshot` (current UTC timestamp, day of week, configured timezone).
- **ConstraintResult**: The output of a single constraint evaluation. Contains an `allowed` boolean, an optional `reason` string, an optional `retry_after` timestamp, an optional `PriceModifier`, and an optional metadata dictionary.
- **PriceModifier**: A value object that encodes a price adjustment. Supports `discount_percent` (0-100), `discount_sats` (flat reduction), `free` (boolean), and `bonus_multiplier` (value > 1.0 grants bonus credits). Modifiers compose: percentages compound multiplicatively, sat discounts add, free propagates (if any modifier sets free=true, the result is free).
- **ConstraintEngine**: The pipeline evaluator. Accepts a list of ToolConstraint instances and an evaluation mode. Modes are:
  - `ALL_MUST_PASS`: Every constraint must return `allowed=true` for the invocation to proceed. PriceModifiers from all passing constraints are accumulated.

- `ANY_MUST_PASS`: At least one constraint must return `allowed=true`.
- `FIRST_MATCH`: The first constraint that returns a definitive result (either allowed or denied with a reason) determines the outcome.

1.8.3.2 3.2 Baked-In Constraint Classes The preferred embodiment includes eleven constraint classes implemented as native code (Python) for performance and reliability:

1. **TemporalWindowConstraint**: Gates tool access to specified wall-clock hours and days of week. Supports timezone specification using the IANA timezone database. Example: tool available Monday-Friday 9:00-17:00 America/New_York. Outside the window, returns `allowed=false` with `retry_after` indicating when the window next opens.

2. **FiniteSupplyConstraint**: Limits total invocations of a tool to a fixed count, either globally (across all consumers) or per-consumer. Example: tool may be invoked at most 1,000 times total, or at most 50 times per consumer. When the supply is exhausted, returns `allowed=false` with reason indicating exhaustion. Global scope reads the lifetime invocation total from the `tool_demand` table (Section 2.7) via the `EnvironmentSnapshot.supply_total_for()` accessor. The lifetime counter is monotonic — Operators raise the `max_invocations` ceiling to grant additional runway without resetting the counter.

3. **PeriodicRefreshConstraint**: Implements rolling-window rate limits with automatic refresh. Specifies a maximum invocation count and a refresh period as an ISO 8601 duration. Example: at most 100 invocations per PT5H (5 hours). The window rolls forward continuously; invocations older than the period are no longer counted.

4. **CouponConstraint**: Validates a promotional code provided by the consumer. Supports configurable expiration dates, maximum total redemptions, and maximum per-consumer redemptions. Valid coupons produce a PriceModifier (discount or free). Expired or exhausted coupons return a neutral result (neither allowing nor denying).

5. **FreeTrialConstraint**: Grants the first N invocations of a tool at zero cost. Never denies — after the trial is consumed, returns a neutral result and the base price applies. Per-consumer tracking. The FreeTrialConstraint replaces earlier hardcoded seed balance grants. Rather than crediting new patrons with a fixed api_sat balance at registration time, the Operator configures a FreeTrialConstraint that grants the first N invocations free. This approach is more flexible — configurable per campaign, per tool, with explicit invocation tracking — and eliminates the need for hardcoded balance initialization logic.

6. **LoyaltyDiscountConstraint**: Applies a discount when the consumer's total historical consumption (total_consumed_api_sats) exceeds a configurable threshold. Rewards sustained usage with reduced pricing.

7. **BulkBonusConstraint**: Applies tiered bonus multipliers based on cumulative consumption levels. Example: after 1,000 api_sats consumed, receive 1.1x bonus; after 10,000, receive 1.25x. Multipliers affect the api_sats credited per purchase, not the Lightning invoice amount.

8. **HappyHourConstraint**: Combines temporal window gating with a discount modifier. Unlike TemporalWindowConstraint, HappyHourConstraint never denies access — it returns a discount during the specified window and a neutral result outside it.
9. **SurgePricingConstraint**: Applies a price multiplier proportional to current demand intensity. The constraint queries the OperatorRuntime's demand tracking system (`get_global_demand()`) and applies a configurable surge multiplier when demand exceeds threshold levels. SurgePricing never denies a request — it only adjusts price.
10. **ExpressionConstraint**: A safe tree-based expression evaluator that operates on the ConstraintContext's fields (ledger balance, patron identity, environment state) using and/or/not/field/op/value syntax. Enables custom pricing logic without code changes.
11. **JsonExpressionConstraint**: An alias for ExpressionConstraint with explicit JSON tree input format.

1.8.3.3 3.3 Extensible Expression Constraint

For operators requiring custom pricing logic beyond the eleven baked-in classes, the system provides a `JsonExpressionConstraint` that evaluates safe expressions specified in JSON configuration.

The expression evaluator uses abstract syntax tree (AST) analysis to ensure safety. It supports comparison operators (==, !=, >, <, >=, <=), logical operators (`and`, `or`, `not`), and field resolution against the ConstraintContext:

- `ledger.balance_api_sats` — current available balance
- `ledger.total_consumed_api_sats` — lifetime consumption
- `ledger.total_deposited_api_sats` — lifetime deposits
- `env.hour` — current hour (0-23) in configured timezone
- `env.day_of_week` — current day (0=Monday through 6=Sunday)
- `patron.id` — consumer identifier

The evaluator explicitly prohibits `eval()`, `exec()`, `compile()`, `import`, attribute access on arbitrary objects, and any operation not in the permitted operator whitelist. A static analysis test verifies these safety properties.

1.8.3.4 3.4 Constraint Configuration

Constraints are defined per-tool in a JSON configuration file loaded at Operator startup. Example:

```
{
  "tools": {
    "search_thoughts": {
      "base_cost_api_sats": 2,
      "constraints": [
        {
          "type": "TemporalWindowConstraint",
          "params": {
            "start_hour": 9, "end_hour": 17,
```

```
            "days": [0, 1, 2, 3, 4],
            "timezone": "America/New_York"
          }
        },
        {
          "type": "FreeTrialConstraint",
          "params": {"free_invocations": 10}
        }
      ],
      "evaluation_mode": "ALL_MUST_PASS"
    }
  }
}
```

A `CONSTRAINT_REGISTRY` maps type strings to constraint classes. Custom constraint classes can be registered by the Operator at startup, enabling arbitrary extensions without modifying the Tollbooth library.

The Constraint Engine exposes two standard MCP tools, registered by `register_standard_tools()`: `check_price` (returns the effective price for a named tool given the current patron's context and active constraints) and `list_constraint_types` (enumerates all registered constraint types with their parameter schemas). These tools enable patrons and pricing management interfaces to inspect pricing in real time.

1.8.3.5 3.5 Runtime-Configurable Pricing Models

In the preferred embodiment, tool pricing and constraint pipelines are not limited to static configuration loaded at process startup. The system provides a runtime-configurable pricing model layer that allows an Operator to define, store, and activate named pricing models without restarting the service process.

A **Pricing Model** is a named bundle comprising:

- **Per-tool price schedule**: A list of `(tool_name, price_sats, category, intent)` tuples that define the deduction cost for each tool. The `category` and `intent` fields are operator-defined metadata that enable grouping and documentation of pricing rationale (e.g., category "read" vs. "write", intent "query" vs. "mutation").
- **Constraint pipeline**: An ordered list of pipeline steps, each referencing a constraint type from the `CONSTRAINT_REGISTRY` along with its configuration parameters. The pipeline is applied as a wildcard constraint (applying to all tools uniformly) and is converted at resolution time into a `ConstraintEngine` instance using the same `load_constraints()` mechanism described in Section 3.4.

Pricing models are persisted in a relational database (PostgreSQL in the preferred embodiment) with the following schema:

- `id` (UUID primary key, auto-generated)
- `operator` (text, the Operator's Nostr npub)
- `name` (text, human-readable model name)

- `model_json` (JSONB, the serialized pricing model document)
- `is_active` (boolean, at most one active model per operator, enforced by a partial unique index)
- `created_at`, `updated_at` (timestamps)

The **at-most-one-active invariant** is enforced at the database level: a partial unique index on `(operator) WHERE is_active = true` ensures that activating a new model atomically deactivates the previous one. This prevents race conditions in concurrent activation requests.

A **Pricing Resolver** provides the runtime integration point. At tool-call time, the resolver returns the effective cost for a tool name and, optionally, a `ConstraintEngine` derived from the active model's pipeline. The resolver maintains an in-memory cache with a configurable time-to-live (default 300 seconds). Cache misses trigger a single database query. Database failures degrade gracefully: the resolver falls back to the cached model (if any) or to a static fallback cost dictionary provided at initialization. This ensures that pricing remains available even during transient database outages.

The resolver exposes a `refresh()` method that resets the cache timestamp, forcing the next cost lookup to query the database. This is the mechanism by which an external pricing management interface (such as a mobile application or administrative dashboard) signals that a new pricing model has been activated.

The `ConstraintGate` (described in Section 3.1) supports an `attach_resolver()` method that wires a Pricing Resolver as a dynamic engine source. When attached, the gate's asynchronous evaluation path (`check_async()`) prefers the resolver's dynamically-loaded constraint engine over the static engine loaded from configuration. If the resolver fails or returns no engine, the gate falls back to the static engine. The synchronous evaluation path (`check()`) is unchanged — no breaking change to existing operator integrations.

This architecture separates three concerns:

1. **Model authoring**: An Operator or a pricing management interface creates and updates pricing models via CRUD operations on the persistent store.
2. **Model activation**: A single activation operation atomically switches the live model, deactivating the previous one.
3. **Model resolution**: The runtime resolver provides sub-millisecond cached lookups with automatic refresh, graceful fallback, and zero-downtime model transitions.

The combination of per-tool cost schedules, composable constraint pipelines, database-enforced activation invariants, TTL-cached resolution, and graceful degradation enables operators to adjust pricing strategy in real-time — responding to market conditions, promotional campaigns, or competitive pressure — without service interruption.

1.8.4 4. Hierarchical Trust Chain with Self-Similar Royalty Distribution (Claim Family 3)

1.8.4.1 4.1 The Self-Similar Pattern The core architectural insight of the trust chain is that an Authority is structurally identical to an Operator. The only positional difference is that an Authority's consumers are other Operators (who call `certify_credits`), while an Operator's

consumers are end users or AI agents (who call domain-specific tools).

This self-similarity means:

- The same Tollbooth middleware runs at every tier
- The same credit purchase flow (Section 2.2) applies at every tier
- The same constraint engine (Section 3) can apply pricing policies at every tier
- The same ledger management (Section 2.4) tracks balances at every tier
- No special-case code exists for "Authority" versus "Operator" — the distinction is purely in which tools are exposed and who the consumers are

This eliminates the need for separate tax collection machinery, separate balance management systems, or separate protocol definitions at different tiers. The Authority's revenue is earned through the `certify_credits` tool fee, not through an architecturally distinct taxation mechanism.

In the preferred embodiment, the self-similar pattern extends to the certification client itself: the same `AuthorityCertifier` class used by an Operator to obtain a certificate from its Authority is also used by that Authority to obtain a certificate from its upstream Authority, automatically, at certification time. This means no tier in the chain requires manual supply management or pre-purchased certificate inventories — each certification request cascades upstream in real-time.

1.8.4.2 4.2 Certification Flow When an Operator wishes to sell credits to its consumers, it must first obtain a signed certificate from its upstream Authority:

1. **Operator requests certification**: The Operator calls the Authority's `certify_credits` tool, specifying the desired purchase amount in satoshis.

2. **Authority verifies standing**: The Authority fetches the community registry (`members.json`) from the version-controlled repository and verifies that the Operator's Nostr npub has `status: "active"`. If the Operator is banned or unregistered, the certification is refused.

3. **Authority calculates fee**: The Authority deducts its certification fee from the requested amount. In the preferred embodiment, the fee is the `certify_credits` tool's configured cost in api_sats, which is denominated in the Authority's own credit system.

4. **Authority signs certificate**: The Authority generates a certificate containing:

   - `gross_sats`: the original requested amount
   - `net_sats`: the amount after fee deduction (what the Operator will receive)
   - `fee_sats`: the Authority's fee
   - `operator_npub`: the requesting Operator's Nostr public key
   - `authority_npub`: the signing Authority's Nostr public key
   - `timestamp`: UTC timestamp of certification
   - `jti`: a unique anti-replay identifier (UUID)
   - `expires_at`: certificate expiration (configurable, default 10 minutes)

   The certificate is signed using the Authority's Schnorr private key (secp256k1 curve, as used in the Nostr protocol and Bitcoin's Taproot). The signature covers the SHA-256 hash of the

canonical JSON serialization of the certificate claims.

5. **Operator receives certificate**: The signed certificate is returned to the Operator as a compact token (base64-encoded JSON with appended signature).

6. **Operator presents certificate to payment system**: When the Operator's `purchase_credits` tool is called by a consumer, the Operator passes the Authority's certificate. The Tollbooth middleware verifies the signature, checks the anti-replay identifier against a local store of used JTIs, validates the expiration, and creates a Lightning invoice for `net_sats` (not `gross_sats`). This ensures the Authority's fee is honored — the Operator cannot invoice for more than the Authority certified.

7. **Anti-replay**: Each certificate's JTI is recorded upon first use. Subsequent attempts to present a certificate with a previously used JTI are rejected. JTI records are pruned after the certificate's maximum possible lifetime (expiration time plus a safety margin).

1.8.4.3 4.3 Chain Topology

The trust chain forms a tree rooted at the First Curator:

```
First Curator (self-signs)
  └── Authority A
        ├── Operator A1
        ├── Operator A2
        └── Sub-Authority A-sub
              ├── Operator A-sub-1
              └── Operator A-sub-2
```

Each Authority obtains its own certificates from its upstream Authority. In the preferred embodiment, this upstream certification is performed automatically and in real-time: when a non-Prime Authority's `certify_credits` tool is invoked by a downstream Operator, the Authority simultaneously calls its upstream Authority's `certify_credits` tool using the same `AuthorityCertifier` client class that Operators use (described in Section 2.2, step 1). This creates a cascading chain of real-time certifications — a single downstream request triggers certification at every tier up to the First Curator.

The First Curator self-signs — this is logically consistent because the First Curator is the root of trust. The self-signed certificate is distinguishable (the `authority_npub` and `operator_npub` fields are the same) but valid. The First Curator's `certify_credits` implementation detects the absence of an upstream Authority address and skips the upstream call.

Revenue flows upward through the chain. Each certification event extracts a fee at each layer. The fee amount at each layer is determined by that layer's `certify_credits` tool pricing — which may itself be subject to constraint evaluation (Section 3). The upstream certificate is included in the response to the downstream caller for audit transparency (`upstream_certificate`, `upstream_jti` fields).

A configuration-time invariant — the **fee floor constraint** — enforces economic viability at every tier. Each non-Prime Authority's local fee rate (expressed as a percentage of the certified amount) must be greater than or equal to its upstream Authority's fee rate. If the local rate is lower than

the upstream rate, the Authority would lose money on every certification — paying more upstream than it collects downstream. The system queries the upstream Authority's published fee rate at startup and rejects the configuration if the invariant is violated, before any certifications occur.

Formally: let `r_local` be the local Authority's certification fee rate (as a percentage) and `r_upstream` be the upstream Authority's fee rate. The invariant requires `r_local >= r_upstream`. Additionally, the local minimum fee floor (`min_fee_sats`) must be greater than or equal to the upstream's minimum fee floor, ensuring that even for small certification amounts where the percentage-based fee rounds to a small value, the absolute fee collected locally is never less than the absolute fee owed upstream.

This invariant is monotonic through the chain: if every layer satisfies `r_local >= r_upstream`, then for any certification cascading from the leaf Operator to the First Curator, each intermediate Authority collects a non-negative margin. The First Curator (which has no upstream) is exempt from this constraint and sets the fee floor for the entire chain.

The fee floor constraint is enforced at the configuration layer, not at the protocol layer. This means the constraint is verifiable by inspection of the Authority's configuration before any economic activity occurs — a fail-fast pattern that prevents operational losses from misconfiguration.

1.8.4.4 4.4 Principal-Agent Separation The system supports separation between the economic actor (Principal) who funds a balance and the operational actor (Agent) who invokes tools. This enables delegation scenarios such as:

- A fleet operator (Principal) funding API access for individual drivers (Agents)
- A parent (Principal) funding educational tool access for a child (Agent)
- A business owner (Principal) provisioning API access for employees (Agents)

In this embodiment, the certificate carries both the Principal's npub and the Agent's npub. The Constraint Engine can evaluate policies based on either identity, enabling Principals to set per-Agent limits (maximum spending, time restrictions, tool access restrictions) through constraint configuration.

1.8.4.5 4.5 Operator Tenant Isolation When an Authority registers a new Operator (Section 4.2), the Authority provisions a dedicated persistence tenant for that Operator within the Authority's own database infrastructure. In the preferred embodiment, using PostgreSQL (via the Neon serverless platform), each Operator receives its own database schema.

The schema name is derived deterministically from the Operator's npub:

1. Compute SHA-256 of the Operator's full npub string
2. Take the first 16 hexadecimal characters of the hash digest
3. Prefix with `op_` to form the schema name (e.g., `op_a3f8c1e2b9d04756`)

This produces a collision-resistant, PostgreSQL-safe identifier. The Authority creates the schema and provisions standard tables within it: a ledger table (consumer credit balances with optimistic concurrency via a version column), a ledger journal (append-only transaction log), a credentials

table (encrypted credential blobs keyed by service and npub), and an anchors table (Bitcoin notarization records). Bitcoin notarization via OpenTimestamps is enabled by default for all Operators; notarization tools (notarize_ledger, get_notarization_proof, list_notarizations) are registered automatically in the standard tool set. The notarization subsystem builds a Merkle tree over all patron ledger balances, submits the root hash to multiple OpenTimestamps calendar servers, and stores the resulting receipts alongside the tree's leaf hashes for independent patron verification. Operators may disable notarization if the operational overhead is undesirable.

The Operator's database connection URL includes a `search_path` parameter set to its schema name, restricting all queries to the Operator's own tables. The Operator has no visibility into other Operators' schemas or the Authority's own administrative tables.

This schema-per-operator model provides:

- **Data isolation**: One Operator cannot read or modify another Operator's consumer ledgers, credentials, or transaction history.
- **Independent lifecycle**: An Operator's schema can be provisioned on registration and dropped on deregistration without affecting any other Operator's data.
- **Shared infrastructure**: All Operators share the same physical database instance, avoiding the cost and operational complexity of per-operator database provisioning while maintaining strict logical separation.
- **Deterministic addressing**: The schema name is computable from the npub alone, requiring no lookup tables or additional state.

The Authority stores the Operator's provisioned configuration (schema name, schema-qualified connection URL) in a `bootstrap_config` table keyed by `(npub, key)`, accessible to the Operator via the `get_operator_config` tool (Section 4.7).

1.8.4.6 4.6 Vault Encryption All ledger data stored in the Operator's tenant is encrypted at the application layer using a key derived from the Operator's private key (nsec). This ensures that the Authority, the database infrastructure provider, and any administrator with database access see only ciphertext.

Key derivation follows HKDF (RFC 5869) with SHA-256:

1. **Extract**: HMAC-SHA256 with a fixed salt (`tollbooth-vault-v1`) and the Operator's nsec bytes as input keying material, producing a pseudorandom key.
2. **Expand**: A single HKDF-Expand round with info string `vault-ledger-encryption` produces a 32-byte AES-256 key.

Encryption uses AES-256-GCM:

1. A random 12-byte nonce is generated per write operation.
2. The plaintext (serialized JSON ledger data) is encrypted with AES-256-GCM, producing ciphertext and a 16-byte authentication tag.
3. The stored value is base64-encoded `nonce || ciphertext || tag`.

Because the nonce is random per write, identical plaintext produces different ciphertext on each store operation, preventing ciphertext comparison attacks. The GCM authentication tag provides

integrity verification — any tampering (bit-flip, truncation, substitution) is detected on decryption.

The encryption is transparent to higher-layer code: the vault's `store_ledger` method encrypts before writing, and `fetch_ledger` decrypts after reading. A heuristic distinguishes legacy plaintext records (JSON beginning with `{` or `[`) from encrypted blobs, enabling transparent migration: the first write to a plaintext record silently upgrades it to encrypted format without data loss or service interruption.

The key isolation property is critical: because the encryption key is derived from the Operator's nsec, and each Operator holds a distinct nsec, no two Operators share an encryption key. Even if a database administrator or infrastructure compromise exposed the raw table contents, the data from Operator A is unreadable without Operator A's nsec, and vice versa.

1.8.4.7 4.7 Operator Bootstrap Protocol An Operator can discover its complete configuration — persistence layer, encryption keys, Authority identity — from a single secret: its Nostr private key (nsec). This eliminates the traditional requirement for environment variable proliferation (database URLs, API keys, Authority endpoints) at deployment time.

The bootstrap sequence:

1. **Identity derivation**: The Operator derives its npub and public key hex from the nsec using standard secp256k1 key derivation.
2. **Authority discovery**: The Operator queries the community registry (Section 6.2) for its own member record, which contains the `upstream_authority_npub` field. It then resolves the Authority's MCP endpoint URL from the Authority's member record.
3. **Configuration retrieval**: The Authority delivers the Operator's provisioned configuration (schema-qualified Neon connection URL) via a NIP-04 encrypted direct message on the Nostr relay network at registration time. The Operator's bootstrap process polls configured relays for this encrypted configuration message. As a fallback, the Operator can also retrieve the configuration by calling the Authority's `get_operator_config` tool. Both paths deliver the same configuration; the DM delivery ensures the Operator receives its configuration even before its first tool call to the Authority.
4. **Vault initialization**: The Operator constructs its `NeonVault` with the retrieved connection URL and its own nsec as the encryption key source.
5. **Onboarding Status Computation**: The Operator computes its onboarding status from three sources: (a) identity readiness (nsec present in environment), (b) authority readiness (bootstrap configuration received from Authority), and (c) operator secret readiness (all fields defined in the `operator_credential_template` are present in the encrypted vault). The `get_onboarding_status` standard tool reports which of these categories are satisfied and provides human-readable remediation paths for any that are missing. This computation is template-driven — the credential template IS the schema — eliminating the need for per-operator Settings class introspection.
6. **Credential delivery**: Missing secrets are delivered via the Secure Courier protocol (Section 5.3).

The bootstrap is optimistic: it first attempts `get_operator_config` (fast path for already-registered Operators). If the Operator is not yet registered, it calls `register_operator` on

the Authority, which provisions the tenant (Section 4.5), then retries the configuration retrieval.

The bootstrap result is cached for the process lifetime. Subsequent tool invocations use the cached vault without repeating the discovery sequence.

This design achieves a deployment model where a new Operator instance requires exactly one environment variable (`TOLLBOOTH_NOSTR_OPERATOR_NSEC`) to join the network. All other configuration is discovered dynamically through the trust chain.

1.8.5 5. Nostr-Based Identity and Credential Exchange (Claim Family 4)

1.8.5.1 5.1 Identity Primitive Every participant in the system is identified by a Nostr public key (npub), a bech32-encoded representation of a secp256k1 public key as defined by the Nostr protocol (NIP-01). The corresponding private key (nsec) is held exclusively by the participant and is never transmitted to, stored by, or accessible to any other system component.

The choice of Nostr keypairs as the identity primitive provides:

- **Portability**: The same npub identifies a participant across all Operators, Authorities, and the community registry. No per-service accounts.
- **Self-sovereignty**: No centralized identity provider. The participant generates their own keypair using standard cryptographic tools.
- **No PII**: Registration requires only an npub — no email address, phone number, legal name, or government identifier.
- **Cryptographic verifiability**: Ownership of an npub is provable through digital signature. The Nostr protocol's event signing mechanism provides this natively.
- **Communication channel**: The same npub that serves as an identifier also provides a messaging address on the Nostr relay network, enabling the Secure Courier protocol (Section 5.3).

1.8.5.2 5.2 Citizenship Verification New participants join the Honor Chain through a cryptographic signature challenge:

1. The applicant calls the Oracle's `request_citizenship` tool, providing their npub and a display name.
2. The Oracle generates a random nonce and returns it along with a challenge ID and signing instructions. The challenge has a 10-minute expiration.
3. The applicant constructs a Nostr event (kind 1, per NIP-01) with content containing the string `DPYC-CITIZENSHIP:<nonce>`, and signs it with their private key.
4. The applicant calls `confirm_citizenship` with the challenge ID, their npub, and the signed event JSON.
5. The Oracle verifies:
    - The challenge exists and has not expired
    - The Schnorr signature is valid per NIP-01
    - The event's pubkey matches the claimed npub
    - The event content contains the issued nonce
    - The npub is not already registered

6. On success, the Oracle commits the new member directly to `members.json` in the community registry repository, with role "citizen", status "active", and upstream_authority_npub set to the First Curator's npub.

This flow proves the applicant controls the private key corresponding to the claimed npub, without the private key ever leaving the applicant's device or being transmitted over any network.

1.8.5.3 5.3 Secure Courier Credential Exchange When an Operator's MCP service requires external API credentials from a consumer (for example, API keys for an upstream service that the MCP tools proxy), the Secure Courier protocol enables privacy-preserving credential delivery using the Nostr relay network:

1. **Channel Opening**: The consumer calls `request_credential_channel`, optionally providing their npub. The Operator's service sends a welcome direct message to the consumer's npub via the Nostr relay network.

2. **Encrypted Delivery**: The consumer replies to the welcome message with their credentials in a structured format. The reply is encrypted using NIP-44 (versioned encryption using XChaCha20-Poly1305 with HKDF-SHA256 key derivation) and wrapped using NIP-17 (gift-wrapped direct messages) for metadata privacy. The NIP-17 wrapping ensures that relay operators cannot determine the sender, recipient, or content of the message.

3. **Credential Receipt**: The consumer calls `receive_credentials`. The service scans configured Nostr relays for encrypted direct messages from the consumer's npub, decrypts the NIP-44 payload, validates the credential format against a service-specific template, and stores the credentials in an encrypted vault.

4. **Vault Storage**: Credentials are encrypted using the same HKDF-SHA256-derived AES-256-GCM key used for ledger encryption (Section 4.6) and stored in the server-side credential vault. The encryption key is derived from the Operator's nsec, which is never stored in the vault — it exists only in the Operator's runtime memory, derived from the deployment-time environment variable.

5. **Anti-Replay**: Each credential delivery includes a "poison nonce" — a random value embedded in the welcome message that must appear in the reply. This binds the credential delivery to a specific channel opening event and prevents replay of old credential messages.

6. **Relay Cleanup**: After successful credential receipt, the encrypted message is deleted from the relay to minimize the exposure window of encrypted credential material on third-party infrastructure.

7. **Ephemeral Agent**: When the consumer's npub is identical to the Operator's npub (self-onboarding), the Secure Courier generates a one-time ephemeral Nostr keypair. The welcome DM is sent from this ephemeral identity, and the consumer replies to it. The ephemeral keypair is discarded after credential receipt. This avoids the silent failure mode where Nostr relays drop self-addressed DMs, and is consistent with NIP-17's ephemeral key pattern for metadata protection.

8. **Credential Card Issuance**: Upon successful credential receipt and vault storage, the Operator constructs a credential card (`ncred1...` encoded token) and delivers it to the patron as a Nostr DM. The ncred card is a portable, self-contained token that the patron can present directly to the Operator to re-authenticate in future sessions without repeating the Secure Courier relay flow.

The bootstrap configuration delivery (Section 4.7) uses NIP-04 encryption (symmetric shared secret from Diffie-Hellman key exchange) for the registration-time Neon URL delivery. The Secure Courier credential exchange uses the more modern NIP-44 encryption with NIP-17 gift-wrapped DM format, providing stronger forward secrecy and reduced metadata leakage.

The Secure Courier ensures that API credentials never appear in the chat interface between the consumer and the AI assistant. They travel on a separate, encrypted channel — conceptually a "diplomatic pouch" — that is inaccessible to the AI model, the chat platform, or any intermediary.

1.8.5.4 5.4 Identity Credentials (Kind 30080) Upon successful credential receipt, the Operator signs a Nostr event of kind 30080 attesting to the patron's relationship with the Operator's service. This identity credential is a replaceable, parameterized event (per NIP-33) that can be verified by any party with access to Nostr relays. It serves as a portable attestation of patron status, usable for cross-Operator trust decisions without exposing the underlying API credentials.

1.8.5.5 5.5 OAuth2 Authorization Flow (Alternative Credential Acquisition) For services that proxy OAuth2-protected APIs (e.g., financial brokerage services), the `patron_credential_template` may specify an OAuth2 authorization flow as an alternative to Secure Courier delivery. The Operator exposes `begin_oauth` and `check_oauth_status` tools. The patron completes a browser-based authorization code grant, and the resulting access and refresh tokens are stored in the Operator's encrypted vault. The Secure Courier path and the OAuth2 path are alternative credential acquisition strategies, selected by the `patron_credential_template` configuration. In both cases, the resulting credentials are stored in the same encrypted vault and accessed through the same runtime methods.

OAuth2 authorization URLs are often hundreds of characters long and difficult for patrons to copy from an AI chat interface. The system provides an Advocate service (Section 6.3) for ephemeral URL shortening: the Operator creates a memorable short slug (e.g., `brave-otter-finds-gold`) that resolves to the full authorization URL. The short URL expires after 24 hours and is stored in a shared database with no tracking or interstitial pages. This URL compression is best-effort; the Operator falls back to the raw URL if the shortening service is unavailable.

1.8.6 6. Network Governance as Economic Defense (Claim Family 5)

1.8.6.1 6.1 Design Philosophy The Tollbooth software is open source under the Apache License 2.0. A bad actor could clone the entire codebase, replace the Authority's wallet address with their own, and collect revenue without participating in the Honor Chain. Software protection is deliberately not pursued. Instead, the system's economic defense resides in the network itself: a cloned Tollbooth operating outside the Honor Chain has no community trust, no Authority chain,

no cross-Operator interoperability, and no access to the Oracle's onboarding and governance services. The clone is a standalone API behind a paywall — possible, but economically uncompetitive with the network.

This design is intentional and draws from network economics: the value of participation in a network grows with the number of participants (Metcalfe's Law), while the cost of replicating the software is fixed. As the network grows, the economic moat widens without any change to the software.

1.8.6.2 6.2 Community Registry

The community registry (`members.json`) is the canonical record of Honor Chain membership. It is stored in a GitHub repository with the following protections:

- **Branch protection**: The `main` branch requires pull requests, at least one approving review from an Authority, passing CI validation, and prohibits force-pushes and branch deletions.
- **CI validation**: Automated checks enforce JSON schema compliance, npub format validation (bech32 encoding, correct length), uniqueness (no duplicate npubs), and upstream reference integrity (every member's `upstream_authority_npub` must reference an existing active member).
- **Git-as-blockchain**: Every commit in the repository hashes its parent in a Merkle tree, forming a tamper-evident chain. Signed commits provide cryptographic attribution. The `git log` serves as a block explorer. The pull request history serves as a consensus record.

Each member record contains:

```
{
  "npub": "npub1...",
  "display_name": "Operator Name",
  "role": "operator",
  "status": "active",
  "member_since": "2026-02-20",
  "upstream_authority_npub": "npub1...",
  "services": [
    {
      "name": "service-name",
      "url": "https://service.example.com/mcp",
      "description": "Service description"
    }
  ]
}
```

1.8.6.3 6.3 Membership Hierarchy

The Honor Chain defines five membership tiers:

Citizen — The entry tier. Requires only a Nostr npub. Verified through the signature challenge (Section 5.2). Citizens can consume API services as consumers and participate in governance discussions. Sponsored by the First Curator by default.

Advocate — A community utility service tier for shared infrastructure that is not individually monetized. Advocates register via the Oracle with a `services[]` array describing their endpoints. Peer MCP servers discover Advocate services through registry-based name resolution (scanning all members for a matching service name), eliminating the need for hardcoded URLs or environment variables. Sponsored by the First Curator. Examples include OAuth2 callback collectors, ephemeral URL shortening services (for compressing long OAuth authorization URLs into human-friendly phrases), and relay aggregators.

Operator — Runs one or more monetized MCP services. Requires sponsorship by an Authority, a BTCPay Server store, and installation of the Tollbooth middleware. Operators collect api_sats from consumer traffic and pay certification fees to their upstream Authority.

Authority — Certifies purchase orders for downstream Operators. Requires sponsorship by an existing Authority (or the First Curator). Collects certification fees. Responsible for vetting onboarding, monitoring downstream conduct, and maintaining network integrity in their region of the chain.

First Curator — The singular root of the trust chain. Identified by a Nostr npub recorded in the community registry under version control. Appoints the initial Authorities, resolves governance disputes, and can issue bans by decree. Succession mechanism for the First Curator role is defined in the governance document and subject to community amendment.

1.8.6.4 6.4 Dispute Resolution and Banning Members who violate community principles (collecting payments without proper certification, storing consumer PII, misrepresenting Authority affiliation, operating cloned Tollbooths outside the Honor Chain) are subject to banning:

1. An Issue is opened on the community repository with evidence of the violation.
2. A 72-hour discussion period allows Authorities to weigh in.
3. If consensus supports banning, a pull request changes the member's status to "banned" with a `ban_reason` linking to the Issue and a `banned_at` timestamp.
4. The pull request is reviewed and merged by an Authority with repository access.

A banned member's `certify_credits` requests are refused by all Authorities that check the registry — cutting off their ability to transact. The member's record remains in `members.json` for transparency and auditability.

Appeals follow the same process: a new Issue, community review, and a restoring pull request if upheld.

1.8.6.5 6.5 Tax Rate Governance The community tax rate (the certification fee percentage) is stored in the community repository and fetched by Operators at deployment time and on periodic refresh. Changes to the tax rate follow the same pull request process as membership changes, providing version-controlled history of all rate modifications.

This mechanism ensures that no single party can unilaterally change the economic terms of the network. The First Curator proposes; the community reviews; the merge constitutes consensus.

1.8.6.6 6.6 Runtime Registry Verification

At the point of certification (Section 4.2, step 2), the Authority performs a real-time check against the community registry:

1. Fetch `members.json` from the repository (with caching and configurable refresh intervals to balance freshness against API rate limits).
2. Look up the requesting Operator's npub.
3. Verify `status == "active"`.
4. If banned or not found, refuse certification with a descriptive error.
5. If active, proceed with certification.

This check is the enforcement mechanism that gives the registry its economic power: a banned member cannot obtain certificates, cannot create invoices, and therefore cannot sell API access through the Honor Chain.

1.8.7 7. AI-Assisted Pricing Campaign Design Tool (Claim Family 6)

The system described in Sections 1–6 provides runtime pricing infrastructure. However, the design of pricing models — selecting per-tool prices, composing constraint pipelines, and projecting revenue — requires domain expertise that most Operators lack. The present section describes a companion tool ("Pricing Studio") that uses AI-guided structured interviews to co-design pricing campaigns with the Operator, validate them against the constraint engine schema, and deploy them to live MCP endpoints.

1.8.7.1 7.1 Structured Interview Architecture

The campaign design process is divided into six sequential interview stages: Inventory, Demand, Value, Cost, Constraints, and Synthesis (herein referred to as "Recommendation"). An AI consultant (implemented as a large language model with a community-managed system prompt) conducts a structured interview, advancing through stages based on information sufficiency rather than rigid turn counts.

Each stage operates as an isolated conversation thread. The AI receives only that stage's messages plus synthesized context from prior stages via a structured insights object. This isolation prevents context contamination between stages and allows the Operator to revisit any prior stage without resending the entire transcript.

The system prompt is fetched from a community-managed repository at session start, with local caching and a bundled fallback. This enables the community to refine interview methodology, calibration examples, and pricing guidance without requiring application updates.

1.8.7.2 7.2 Machine-Parseable Progress Tracking

Each AI response includes a hidden machine-parseable progress block (`<!-- PROGRESS {...} -->`) containing the current stage identifier, stage number, and a cumulative insights object. The application parses this block to drive a visual stage stepper, populate insight cards, and determine when stage transitions occur. A separate `<!-- REVENUE {...} -->` block in the Synthesis stage provides structured revenue projections (TAM/SAM/SOM analysis and three-scenario forecasts) that the application renders as visual tables and charts.

Upon Operator approval, the AI emits a `<!-- CAMPAIGN_JSON {...} -->` block containing the complete pricing model as structured JSON — tool prices, categories, intents, and constraint pipeline steps. This JSON is extracted by the application for validation, comparison, and deployment. All machine blocks are stripped before display; the Operator sees only rendered markdown tables and prose.

1.8.7.3 7.3 Client-Side Pipeline Validation and Repair Before a pricing model is submitted to the server, the application performs client-side validation against a local constraint catalog that mirrors the server's `CONSTRAINT_REGISTRY`. For each pipeline step, the validator:

1. Verifies the constraint type is known.
2. Checks that all required parameters are present.
3. Backfills missing optional parameters with catalog defaults (e.g., defaulting `happy_hour.schedule` to `"17:00-19:00"` or `temporal_window.timezone` to `"UTC"`).
4. Classifies validation failures as warnings (repairable with defaults) or hard errors (missing required parameter with no default).

Hard errors block the save operation. Warnings are displayed to the Operator but allow the save to proceed. This two-tier validation catches mismatches between the AI's proposed parameters and the server's constraint schema — a category of error that arises when the community prompt's constraint documentation diverges from the deployed constraint engine.

1.8.7.4 7.4 Multi-Provider Second Opinion After the interview reaches the Synthesis stage, the Operator may request a "second opinion" from an independent AI provider. The application constructs a campaign summary (interview insights, revenue projections, proposed pricing JSON, and a condensed consultant transcript) and submits it to a different LLM provider (e.g., xAI Grok when the primary consultant is Anthropic Claude, or vice versa).

The reviewer operates under a separate community-managed system prompt that instructs it to critique the campaign within the DPYC economic framework — not from conventional SaaS or enterprise pricing norms. The review is structured into five sections: Strengths, Risks and Weaknesses, Alternative Pricing Suggestions, Revenue Impact Assessment, and Final Verdict.

Upon dismissal of the review, the application automatically feeds the reviewer's suggestions back to the primary consultant as a follow-up message, enabling the consultant to adjust the proposal based on the peer review. This cross-provider adversarial review process reduces single-model bias in pricing recommendations.

1.8.7.5 7.5 Campaign Lifecycle: Design, Review, Compare, Deploy The campaign design tool supports a full lifecycle:

1. **Design** — AI-guided structured interview produces a pricing model JSON.
2. **Review** — Multi-provider second opinion with structured critique.
3. **Compare** — Side-by-side A/B/C variant comparison with revenue projections, differential analysis, and winner identification.
4. **Deploy** — The approved campaign is pushed to the Operator's live MCP endpoint via a `set_pricing_model` tool call, authenticated by a NIP-98 operator proof (Section 5,

kind 27235 Nostr event). The server validates the pipeline against its constraint registry and atomically activates the new model, deactivating any prior active model.

Campaigns are persisted locally as SwiftData entities with per-stage message storage, revenue projections, second opinion text, and the extracted pricing JSON. Multiple campaigns can coexist; the Operator selects which to deploy.

1.8.7.6 7.6 Operator Identity Proof for Deployment Deploying a pricing model is a RESTRICTED operation — only the Operator whose npub matches the MCP endpoint may modify its pricing. The deployment tool constructs a NIP-98 HTTP Auth event (Nostr kind 27235), signs it with the Operator's nsec (stored in the device Keychain), and embeds the signed event in the `set_pricing_model` tool call arguments. The server verifies the signature, confirms the npub matches the authenticated session, and rejects the operation if verification fails. This ensures that pricing models can only be deployed by their rightful Operator, using the same Nostr identity system described in Section 5.

1.9 Abstract

A system and method for monetizing tool-based application programming interfaces (APIs) through pre-funded credit balances, composable pricing constraints, hierarchical trust chains, decentralized cryptographic identity, community governance, and AI-assisted pricing campaign design.

API consumers pre-fund credit balances denominated in internal units (api_sats) via Lightning Network micropayments. Individual tool invocations deduct from the balance according to pricing rules evaluated by a composable Constraint Engine supporting temporal windows, supply caps, rate limits, surge pricing, promotional modifiers, and custom expressions. Pricing models — named bundles of per-tool costs and constraint pipelines — are runtime-configurable, persisted in a database, and resolved at tool-call time through a TTL-cached resolver with graceful fallback.

Revenue is distributed through a hierarchical trust chain in which Authorities certify purchase orders for downstream Operators using Schnorr digital signatures. Each layer in the hierarchy implements the same protocol pattern — an Authority is structurally an Operator whose consumers are other Operators. A configuration-time fee floor invariant ensures that each Authority's local fee rate meets or exceeds its upstream rate, preventing economically unviable configurations. Authorities provision isolated database schemas for each Operator; Operators encrypt all persisted data with keys derived from their own private keys via HKDF-SHA256. A bootstrap protocol enables Operators to discover their full configuration from a single private key. Community utility services (Advocates) provide shared infrastructure discoverable via registry-based service name resolution.

Participants are identified by Nostr public keys (npubs). Citizenship is verified through cryptographic signature challenges. API credentials are exchanged through encrypted direct messages (NIP-44, NIP-17) on the Nostr relay network, eliminating the need for email, passwords, OAuth tokens, or personally identifiable information. Portable credential cards (ncred) and operator-signed identity credentials (Nostr kind 30080) provide redeemable, verifiable attestations of patron relationships.

The system is governed through a version-controlled community registry on a collaborative source code platform, providing cryptographic auditability through Merkle-tree commit history. Economic defense resides in network participation rather than in proprietary code.

Pricing models are designed through an AI-assisted campaign tool that conducts structured six-stage interviews with Operators, validates proposed constraint pipelines against the server's schema with automatic repair, solicits adversarial second opinions from independent AI providers, and deploys approved campaigns to live MCP endpoints authenticated by Nostr cryptographic operator proofs.

1.10 Appendix A: Implementation References

The preferred embodiment is implemented across the following open-source repositories and deployed services:

Component	Repository	Purpose
Tollbooth Library	`tollbooth-dpyc` (PyPI)	Operator-side middleware: credit management, constraint engine, certificate verification, Lightning integration
Authority Service	`tollbooth-authority`	Authority-side MCP service: purchase certification, registry verification, Schnorr signing
Oracle Service	`dpyc-oracle`	Community onboarding, governance, member lookup, citizenship verification
Community Registry	`lonniev/dpyc-community`	`members.json`, GOVERNANCE.md, tax rate, network status
Reference Operator	`lonniev/thebrain-mcp`	First Tollbooth-monetized MCP service (TheBrain knowledge graph API)
Reference Operator	`lonniev/excalibur-mcp`	Second Tollbooth-monetized MCP service (X/Twitter posting)
Reference Operator	`lonniev/schwab-mcp`	Third Tollbooth-monetized MCP service (Charles Schwab brokerage)

Component	Repository	Purpose
Reference Operator	`lonniev/tollbooth-sample`	Educational reference Operator (weather data, Open-Meteo) with onboarding template
Campaign Design Tool	`lonniev/pricing-studio`	AI-assisted pricing campaign designer (iPad/iOS SwiftUI application, Section 7)
Community Prompts	`lonniev/dpyc-community/prompts/`	Community-managed system prompts for AI consultant and peer reviewer

The prior art GPG-signed tag `v1.0.0-prior-art` (dated February 16, 2026) on the `lonniev/thebrain-mcp` repository establishes the earliest cryptographic proof of the system's existence. The repository was made publicly accessible on February 17, 2026.

1.11 Appendix B: Cryptographic Primitives

Primitive	Usage	Specification
secp256k1 Schnorr signatures	Certificate signing, Nostr event signing, citizenship verification	BIP-340, NIP-01
SHA-256	Certificate claim hashing, Merkle tree (git)	FIPS 180-4
XChaCha20-Poly1305	NIP-44 payload encryption (Secure Courier)	RFC 8439 (extended nonce variant)
HKDF-SHA256	NIP-44 key derivation from ECDH shared secret; vault encryption key derivation from operator nsec	RFC 5869
AES-256-GCM	Vault ledger encryption (random nonce per write, authenticated)	NIST SP 800-38D

Primitive	Usage	Specification
NIP-17 gift wrapping	Metadata-private message delivery	Nostr NIP-17
bech32 encoding	Nostr npub/nsec representation	BIP-173, NIP-19
Lightning Network BOLT-11	Invoice generation and settlement	BOLT #11

1.12 Appendix C: Constraint Engine Class Hierarchy

```
ToolConstraint (ABC)
├── TemporalWindowConstraint
├── FiniteSupplyConstraint
├── PeriodicRefreshConstraint
├── CouponConstraint
├── FreeTrialConstraint
├── LoyaltyDiscountConstraint
├── BulkBonusConstraint
├── HappyHourConstraint
├── SurgePricingConstraint
├── ExpressionConstraint
└── JsonExpressionConstraint (alias for ExpressionConstraint)
```

ConstraintEngine evaluation modes: - `ALL_MUST_PASS`: Conjunction — all constraints must allow - `ANY_MUST_PASS`: Disjunction — at least one must allow - `FIRST_MATCH`: Short-circuit — first definitive result wins

PriceModifier composition rules: - Percentage discounts compound multiplicatively: 10% + 20% = 28% total discount - Satoshi discounts add: 5 sats off + 3 sats off = 8 sats off - Free propagates: if any modifier sets free=true, result is free - Bonus multipliers compound multiplicatively: 1.1x * 1.25x = 1.375x - Surge multipliers compound multiplicatively: 1.2x * 1.5x = 1.8x

1.13 Appendix D: Runtime Pricing Model Architecture

```
PricingModel
├── model_id (UUID)
├── operator (npub)
├── name (human-readable)
├── is_active (boolean, at-most-one-per-operator)
├── tools: list[ToolPrice]
│      └── (tool_name, price_sats, category, intent)
└── pipeline: list[PipelineStep]
       └── (id, type → CONSTRAINT_REGISTRY, params)

PricingModelStore (Neon CRUD)
```

```
├── create_model() → UUID
├── list_models(operator) → list[PricingModel]
├── fetch_active_model(operator) → PricingModel | None
├── update_model(model_id, model_json)
├── activate_model(model_id, operator)    [deactivate-then-
activate]
└── delete_model(model_id)  [refuses active models]

PricingResolver (runtime cache)
├── get_cost(tool_name) → int  [model → fallback → 0]
├── get_constraint_engine() → ConstraintEngine | None
└── refresh()  [force cache reset]

ConstraintGate (integration)
├── check()  [synchronous, static engine — unchanged]
├── check_async()  [async, prefers resolver engine → fallback to static]
└── attach_resolver(resolver)  [wire dynamic source]
```

Database invariant: Partial unique index `(operator) WHERE is_active = true` ensures at most one active pricing model per operator at the database level.

End of Provisional Patent Application Specification Draft

Prepared by: Lonnie VanZandt (Inventor) with AI assistance (Claude, Anthropic) Date: March 2026 Status: DRAFT — Requires review by registered patent attorney or agent